資料分析與機器學習 (Data Analysis and Machine Learning)

卷積神經網路 (CNN) 實質為影像特徵提取器，透過稀疏連接與參數共享降低處理影像的 DNN 之參數需求量

Lecture 8 – Convolutional Neural Networks (CNN) & PyTorch

教師：陳慶永

Outline

- Convolutional Neural Network (CNN)
  - Convolution
    - Kernel, stride, padding
  - Pooling
  - Dropout
- Data Augmentation
- Building and Training Deep CNNs in PyTorch
  - Handwritten digit recognition
  - Smile classification from face images
- Classwork
  - Assignment #4

Convolutional Neural Network (CNN)

大腦的啟發：1959 年 Hubel 與 Wiesel 透過貓的初級視覺皮層實驗，發現視覺細胞具備分工特性。前層細胞主要負責辨識「邊緣與直線」等基礎特徵，後層細胞則負責將其組合為「複雜形狀與輪廓」

Understanding CNN

- Successfully extracting relevant features is key to the performance of any ML algorithm
- CNN layers, as feature extractors, can *automatically* learn useful features from raw data
  - Early layers extract low-level features from raw data
    - Edges and blobs
  - Later layers (often fully connected layers, as in a MLP) use these features to predict a continuous target value or class label

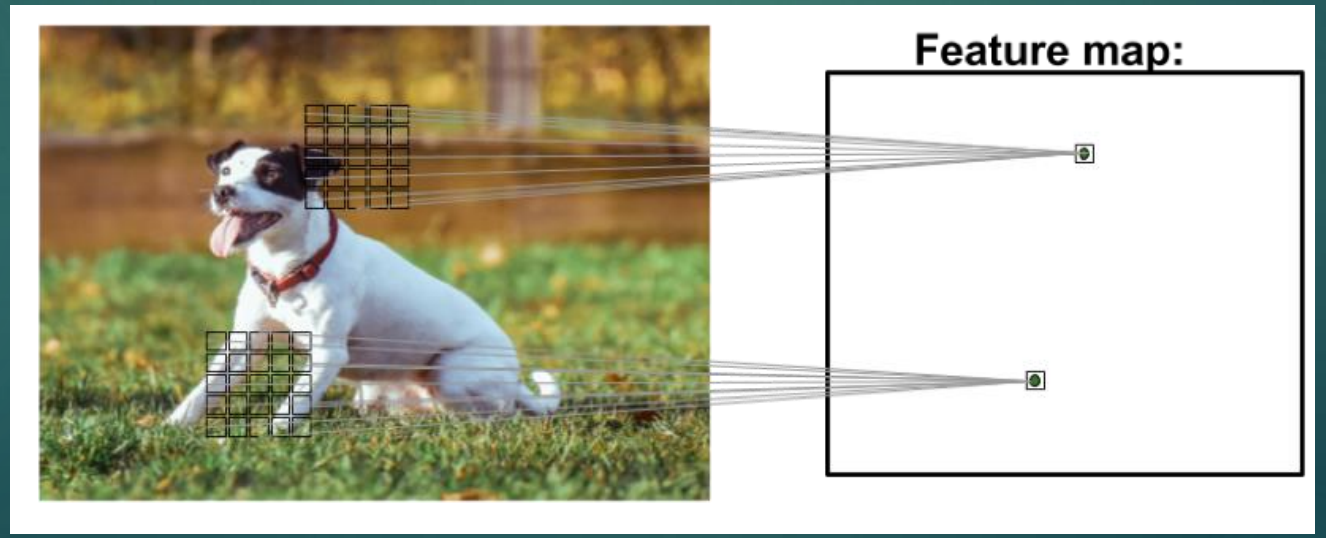
傳統機器學習極度依賴領域專家手工設計特徵 (如 HOG、SIFT)。CNN 則能從原始數據中「自動學習」最具代表性的特徵

Understanding Deep CNN

- Feature Hierarchies
  - *Deep* CNNs, construct a feature hierarchy by combining low-level features in a layer-wise fashion to form high-level features
    - High-level features can form more complex shapes
      - Contours of objects like buildings, cats or dogs
  - Convolutional layers are often described as feature extraction layers

Feature Map (特徵圖)

- CNN computes feature maps from an input image
  - Each element comes from a local patch of pixels in the input image
    - This local patch of pixels is referred to as the local receptive field (局部感知域)



Ideas behind CNN

- CNNs perform well on image-related tasks due to two key ideas
  - Sparse connectivity (稀疏連接)
    - A single element in the feature map is connected to *only* a small patch of pixels (Very different from connecting to the *whole* input image as in MLPs)
  - Parameter sharing (參數共享)
    - The *same weights* are used for different patches of the input image (Translation Invariance (平移不變性))

Ideas behind CNN

- Replacing a fully connected layer with a convolution layer substantially decreases the number of weights (parameters)

```
傳統 MLP（全連接）：  [所有輸入像素] ——（獨立權重）——> [隱藏層單元]（參數爆炸）
CNN 卷積層（局部）：  [局部感受野] ——（共享卷積核）——> [特徵圖單元]（參數極少）
```

  - Make sense for image data to assume that nearby pixels are typically more relevant to each other than pixels that are far away from each other (鄰近像素通常比遠處像素更具備高度相關性)

傳統全連接網路 (MLP) 處理影像時，會因「像素全連接」導致參數爆炸 (例如 1024×1024 的彩色影像會產生百萬級輸入單元)

Ideas behind CNN

- Typically, CNNs are composed of several convolutional and subsampling (pooling) layers that are followed by one or more fully connected layers at the end
  - The fully connected layers are essentially an MLP where every input unit is connected to every output unit via weight w_{ij}

1D Discrete Convolution

- Definition of discrete convolution
  - Output $\boldsymbol{y}$
  - Input $\boldsymbol{x}$
    - Finite length vectors in practice
  - Kernel (卷積核) (filter) $\boldsymbol{w}$
    - Finite length vectors in practice

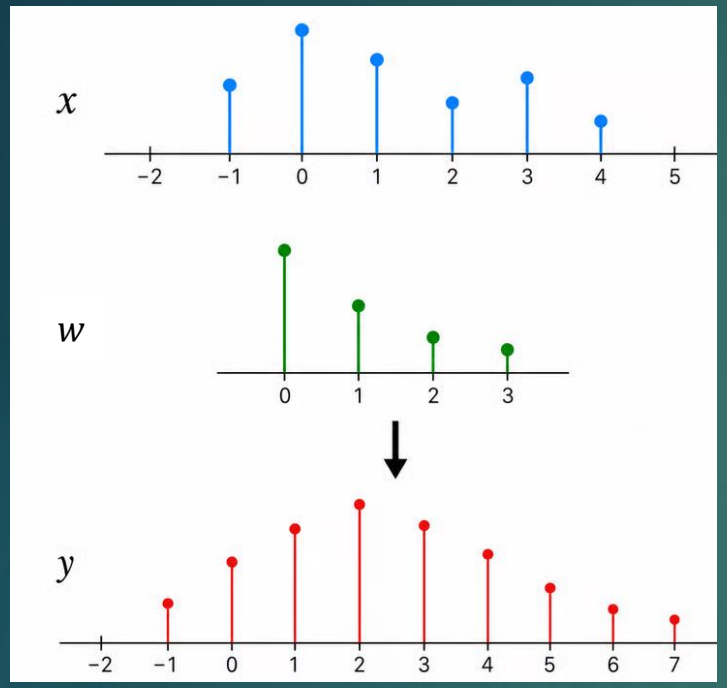
$$\boldsymbol{y} = \boldsymbol{x} * \boldsymbol{w} \rightarrow y[i] = \sum_{k=-\infty}^{+\infty} x[i-k]\, w[k]$$

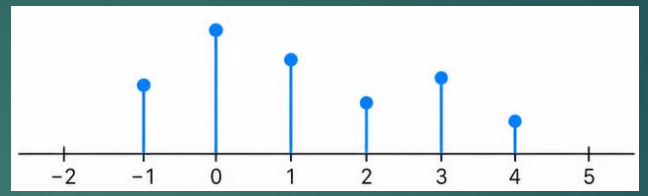
$$= \sum_{k=-\infty}^{+\infty} x[k] w[i-k]$$

(commutativity)

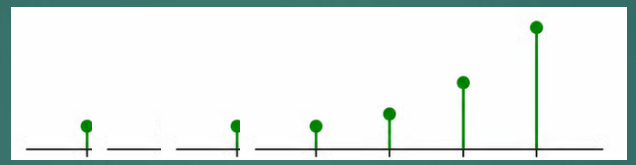
- Note that $\boldsymbol{x}$ and $\boldsymbol{w}$ are indexed in different directions in this summation

1D Discrete Convolution

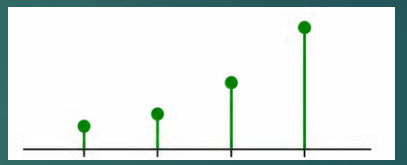




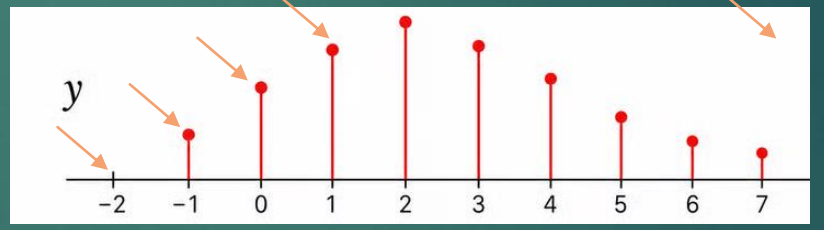
Element-wise
Product & Summation



$w[-2 \; w[k] \; 1 - w[-k] \; w[1 - k]$
$= w[-(k \; w[-2 \; 0 \; +1)]$ $y[-(k-1)]$



$w[7-k]$
$= w[-(k-7)]$



$y[-2]y[-1]y[0] \quad y[1] \qquad\qquad y[7]$

Cross-correlation

- Most deep learning frameworks (including PyTorch) implement cross-correlation but refer to it as convolution

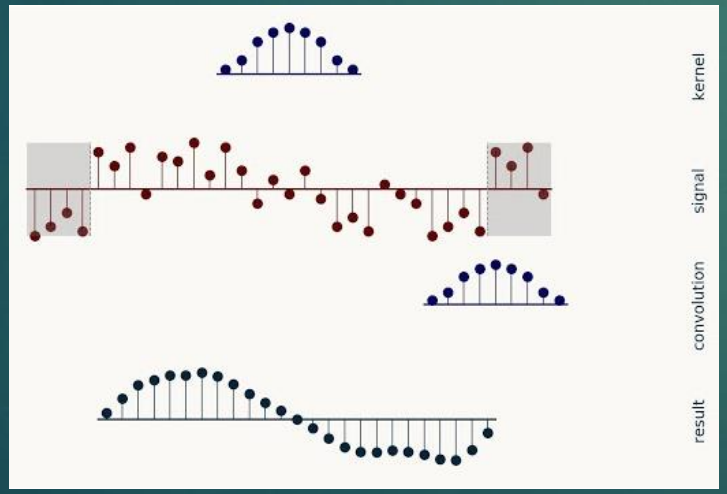
$$\boldsymbol{y} = \boldsymbol{x} \star \boldsymbol{w} \rightarrow y[i] = \sum_{k=-\infty}^{+\infty} x[i+k]\, w[k]$$

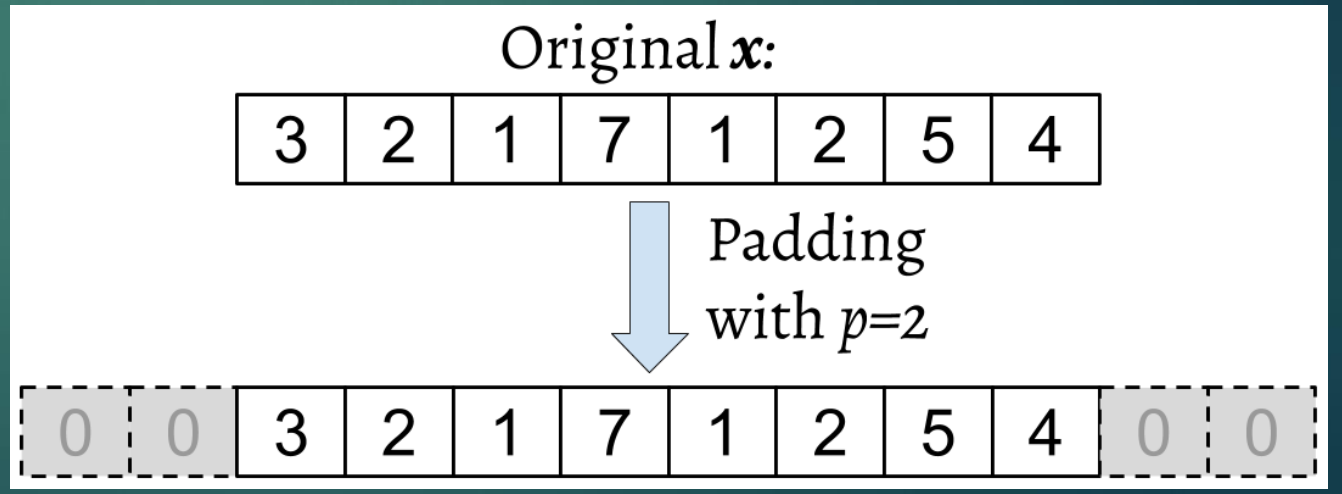
  - The distinction is that *convolution requires flipping the filter before computing dot products*

技術上深度學習框架 (如 PyTorch) 實作的其實是 Cross-correlation，即不對卷積核進行旋轉，這在深度學習領域已約定俗成統稱為卷積

Zero-padding (or Padding) (填充)

- To correctly compute the convolution, it is assumed that $\boldsymbol{x}$ and $\boldsymbol{w}$ are filled with zeros
  - $\boldsymbol{x}$ is padded only with a finite number of zeros
    - The number of zeros padded *on each side* is denoted by p





Zero-padded Convolution

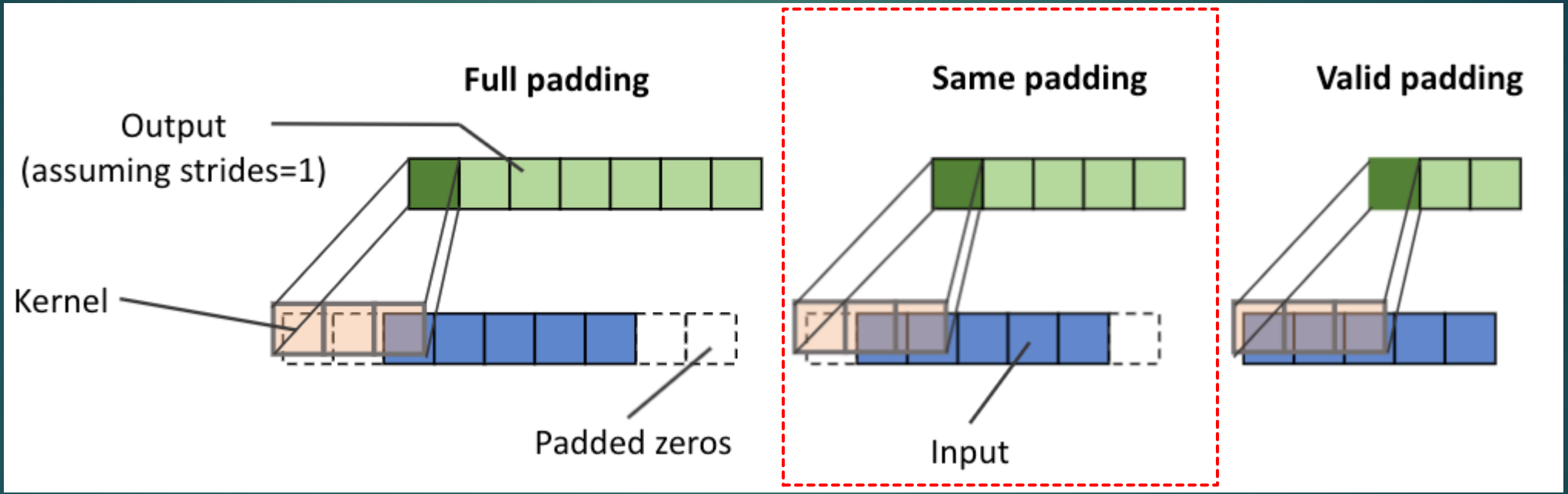
- Assume that the original input $\boldsymbol{x}$ have n elements
  - The *padded* vector $\boldsymbol{x}^p$ has size $n + 2p$
- Assume that the filter $\boldsymbol{w}$ have m elements, $m \leq n$

$$\boldsymbol{y} = \boldsymbol{x} * \boldsymbol{w} \rightarrow y[i] = \sum_{k=0}^{k=m-1} x^p[i + m - k]\, w[k]$$

Padding Modes

- Full Mode
  - $p = m - 1$
  - The output is larger than the input (rarely used in ML)
- Same Mode
  - p is dynamically computed based on filter size so that output spatial dimensions exactly match input dimensions
    - Make designing a network architecture more convenient
  - The most commonly used padding mode in CNN
- Valid Mode
  - $p = 0$ (no padding)
  - Tensor volume drops rapidly across deep layers

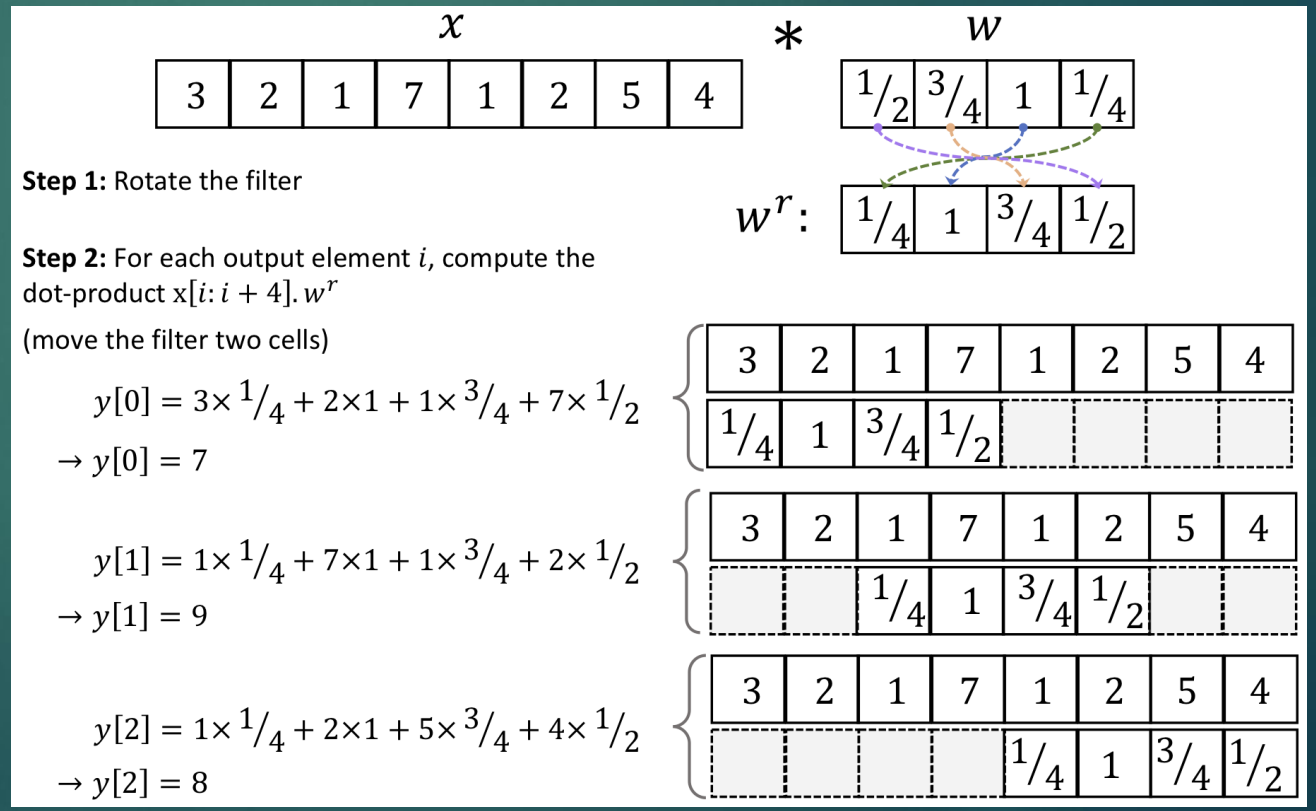
Padding Modes



Stride (步長)

- The number of cells the filter shifts at each step
  - It must be a positive integer smaller than the input size
  - Example:
    - Kernel $m = 4$
    - Padding $p = 0$
    - Stride $s = 2$

Hyperparameters



Size of the Convolution Output

- Determined by the total number of times we shift the filter
  - n: input size
  - m: filter size
  - p: padding
  - s: stride

$$o = \left\lfloor \frac{n + 2p - m}{s} \right\rfloor + 1$$

- In practice, we preserve the spatial size using "same padding" for the convolutional layers
  - Decrease the spatial size via pooling layers (or convolutional layers with stride 2 instead)

$$n = 10, m = 5, \quad p = 2, \quad s = 1 \rightarrow o = \left[\frac{10 + 2 \times 2 - 5}{1} \right] + 1 = 10$$

2D Discrete Convolution

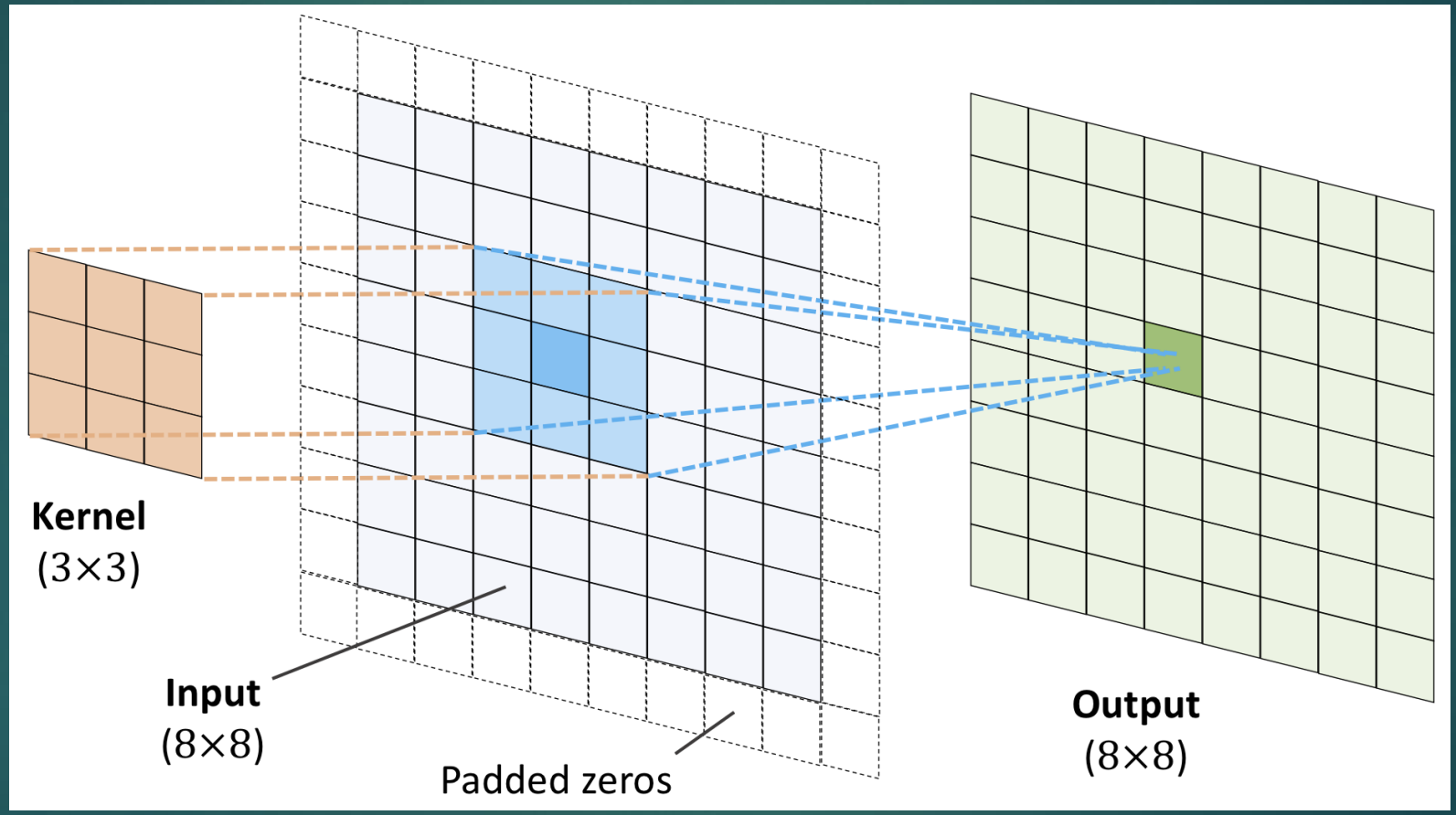
- Definition
  - Output $\boldsymbol{Y}$
  - Input $\boldsymbol{X}$
    - Finite size matrices in practice
  - Kernel (filter) $\boldsymbol{W}$
    - Finite size matrices in practice

$$\boldsymbol{Y} = \boldsymbol{X} * \boldsymbol{W} \rightarrow Y[i,j] = \sum_{k_1=-\infty}^{+\infty} \sum_{k_2=-\infty}^{+\infty} X[i-k_1, j-k_2]\, W[k_1, k_2]$$

Element-wise Product & Summation

- Zero padding, rotating the filter matrix, and the use of strides are also applicable to 2D convolutions (extended to both dimensions independently)

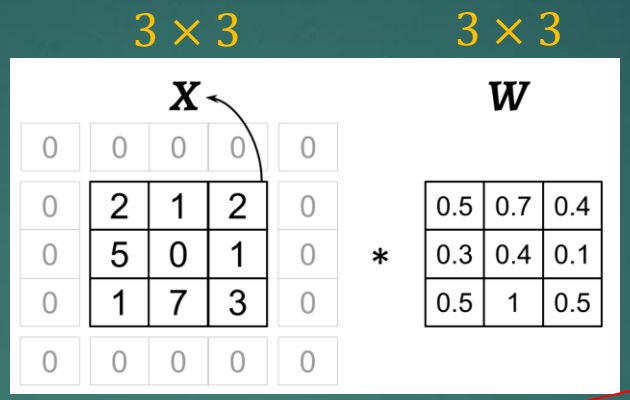
2D Discrete Convolution



Modern CNNs usually use kernel sizes such as 1×1, 3×3, or 5×5

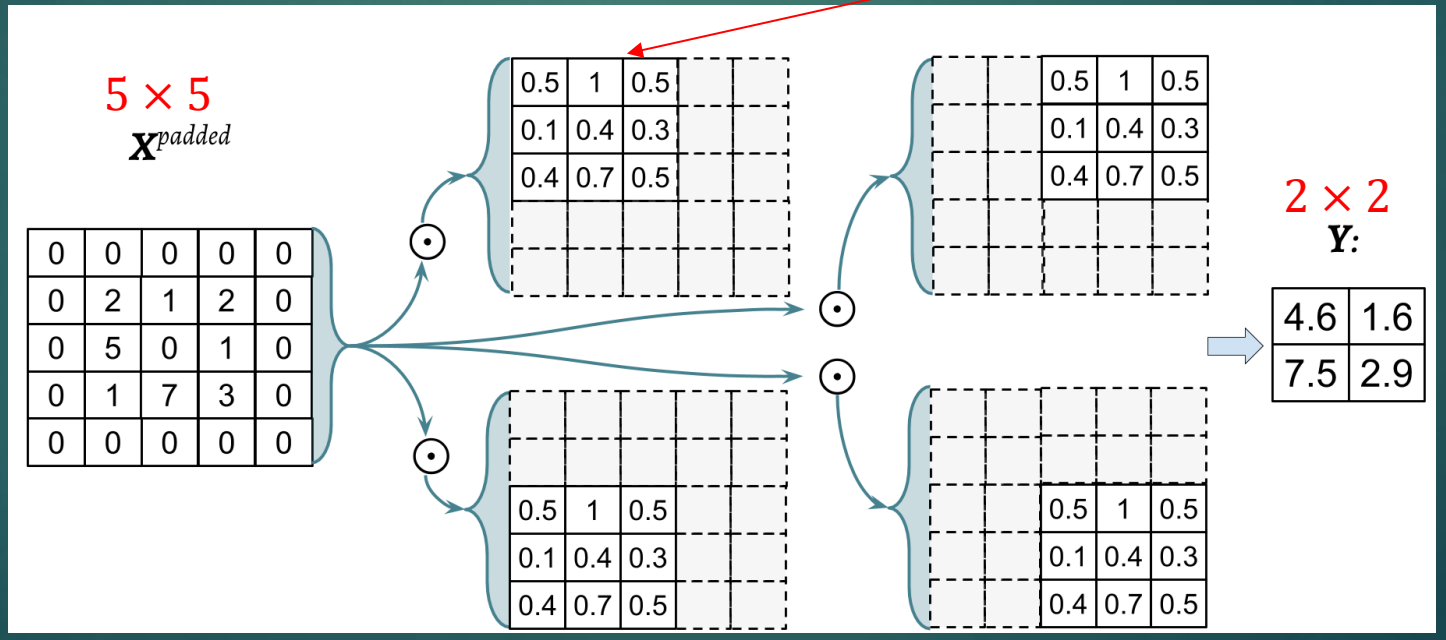
2D Discrete Convolution

- Example:
  - Padding $p = (1,1)$
  - Stride $s = (2,2)$



Rotated filter

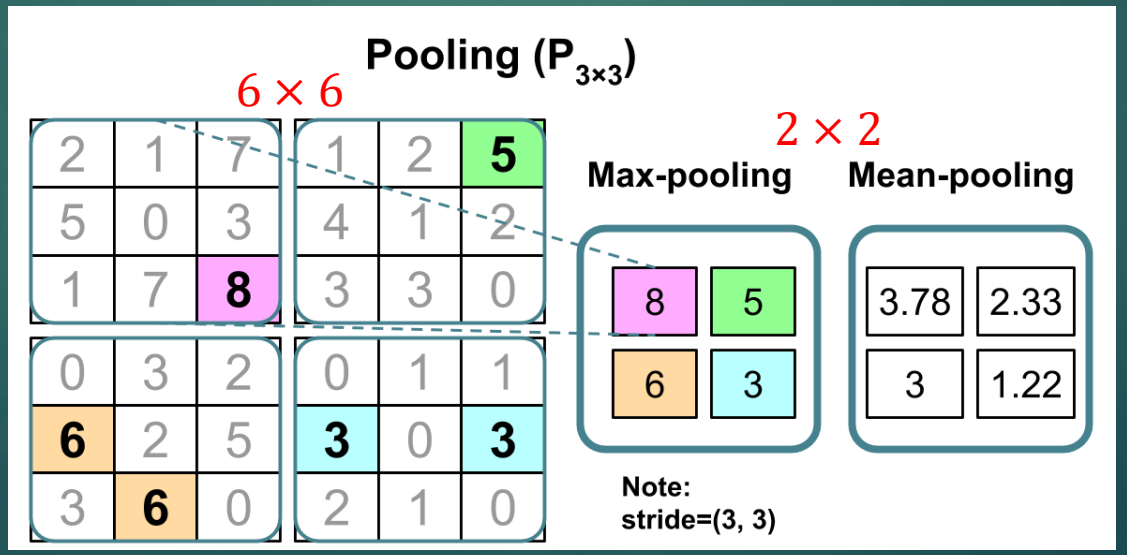
$$\boldsymbol{W}^r = \begin{bmatrix} 0.5 & 1 & 0.5 \\ 0.1 & 0.4 & 0.3 \\ 0.4 & 0.7 & 0.5 \end{bmatrix}$$



Note that the filter matrix is actually not rotated in most tools like PyTorch

Subsampling (Pooling) Layers

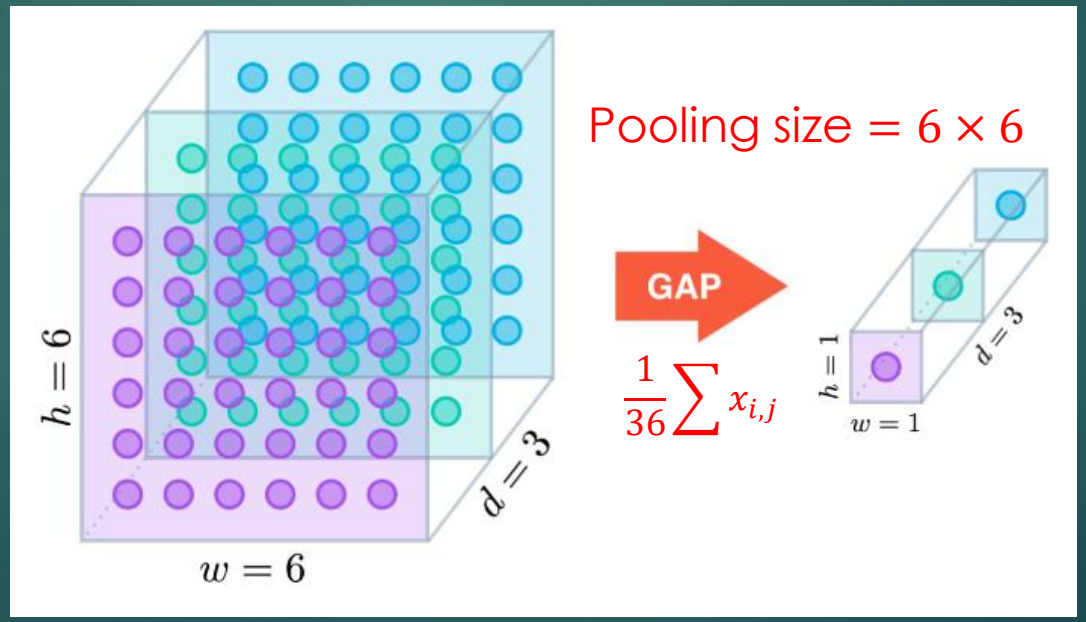
- Pooling layer (池化層) is usually denoted by $P_{n_1 \times n_2}$
  - Pooling size: $n_1 \times n_2$ (size of the neighborhood)
  - Two forms of operations
    - Max-pooling (最大池化)
    - Mean-pooling (or average-pooling) (平均池化)



Non-overlapping pooling

Subsampling (Pooling) Layers

- Global average-pooling (GAP) (全域平均池化)
  - A special case of average-pooling when the pooling size is equal to the size of the input feature maps



Why Pooling?

- Introduces a local invariance
  - Small changes in a local neighborhood do not change the result (of max-pooling)
  - More robust to noise in the input data
- Decreases the size of features
  - Higher computational efficiency
  - May reduce the degree of overfitting

$$X_1 = \begin{bmatrix} 10 & 255 & 125 & 0 & 170 & 100 \\ 70 & 255 & 105 & 25 & 25 & 70 \\ 255 & 0 & 150 & 0 & 10 & 10 \\ 0 & 255 & 10 & 10 & 150 & 20 \\ 70 & 15 & 200 & 100 & 95 & 0 \\ 35 & 25 & 100 & 20 & 0 & 60 \end{bmatrix}$$

$$X_2 = \begin{bmatrix} 100 & 100 & 100 & 50 & 100 & 50 \\ 95 & 255 & 100 & 125 & 125 & 170 \\ 80 & 40 & 10 & 10 & 125 & 150 \\ 255 & 30 & 150 & 20 & 120 & 125 \\ 30 & 30 & 150 & 100 & 70 & 70 \\ 70 & 30 & 100 & 200 & 70 & 95 \end{bmatrix}$$

$$\xrightarrow{\text{max pooling } P_{2\times 2}} \begin{bmatrix} 255 & 125 & 170 \\ 255 & 150 & 150 \\ 70 & 200 & 95 \end{bmatrix}$$

Stride $s = (2,2)$

Non-overlapping pooling

Overlapping pooling occurs if the stride is smaller than the pooling size

Alternative Subsampling

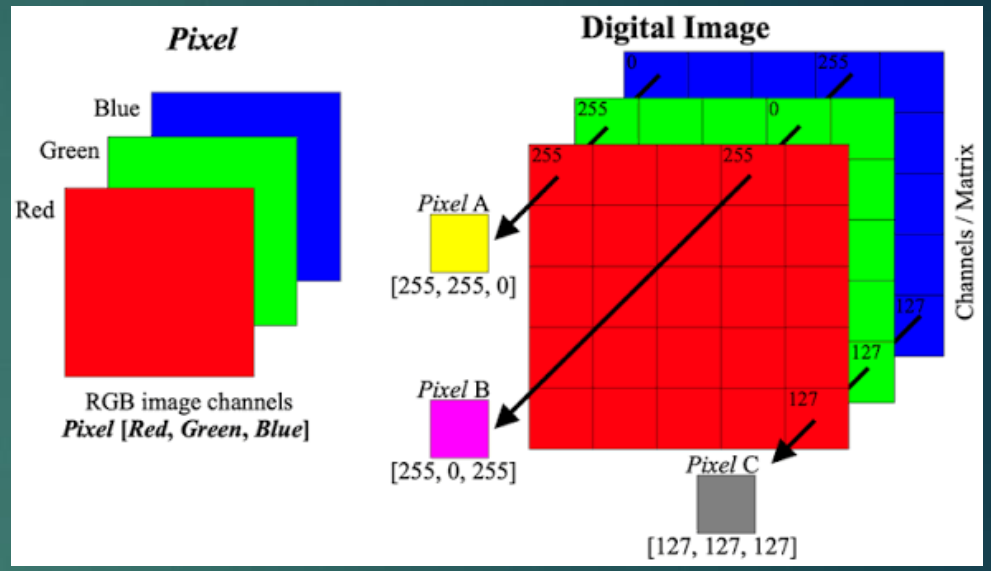
- Note that subsampling (pooling) layers do not have any learnable parameters
- Instead of using pooling layers to reduce the feature size, researchers use convolutional layers with a stride of 2
  - Treat it as a pooling layer *with learnable weights*

Input with Multiple Channels

- 3D array $\boldsymbol{X}_{H\times W\times C}$
  - H: image height (pixels)
  - W: image width (pixels)
  - C: number of input channels
    - Examples:
      - Gray-scale image: $C = 1$
      - RGB color image: $C = 3$ (red, green and blue channels)
  - $\boldsymbol{X}_{H\times W\times C}$ is represented by a rank-3 tensor
    - In torchvision, image tensors are in the format

      Tensor[channels, image_height, image_width]

      C H W




```
>>> import torch
>>> from torchvision.io import read_image
>>> img = read_image('example-image.png')
>>> print('Image shape:', img.shape)
Image shape: torch.Size([3, 252, 221])
>>> print('Number of channels:', img.shape[0])
Number of channels: 3
```

Input with Multiple Channels

- Incorporate multiple input channels in convolution
  - Perform the *convolution for each channel separately*
    - The convolution associated with each channel (c) has its own kernel matrix as $W[:,:,c]$
  - Then *add the results* together using the matrix summation

Given an example $\boldsymbol{X}_{n_1 \times n_2 \times C_{in}}$, a kernel matrix $\boldsymbol{W}_{m_1 \times m_2 \times C_{in}}$, and a bias value b

$$\Rightarrow \begin{cases} \boldsymbol{Z}^{Conv} = \sum_{c=1}^{C_{in}} \boldsymbol{W}[:,:,c] * \boldsymbol{X}[:,:,c] \\ \text{Pre-activation:} \quad \boldsymbol{Z} = \boldsymbol{Z}^{Conv} + b_c \\ \text{Feature map:} \quad \boldsymbol{A} = \sigma(\boldsymbol{Z}) \end{cases}$$

Kernel Input

Output

Output with Multiple Channels

- Typically, a convolutional layer has more than one feature map (output)
  - 4D kernel tensor: $H \times W \times C_{in} \times C_{out}$
    - C_{out} is the number of output feature maps

Given an example $\boldsymbol{X}_{n_1 \times n_2 \times C_{in}}$,
a kernel matrix $\boldsymbol{W}_{m_1 \times m_2 \times C_{in} \times C_{out}}$,
and a bias vector $\boldsymbol{b}_{C_{out}}$

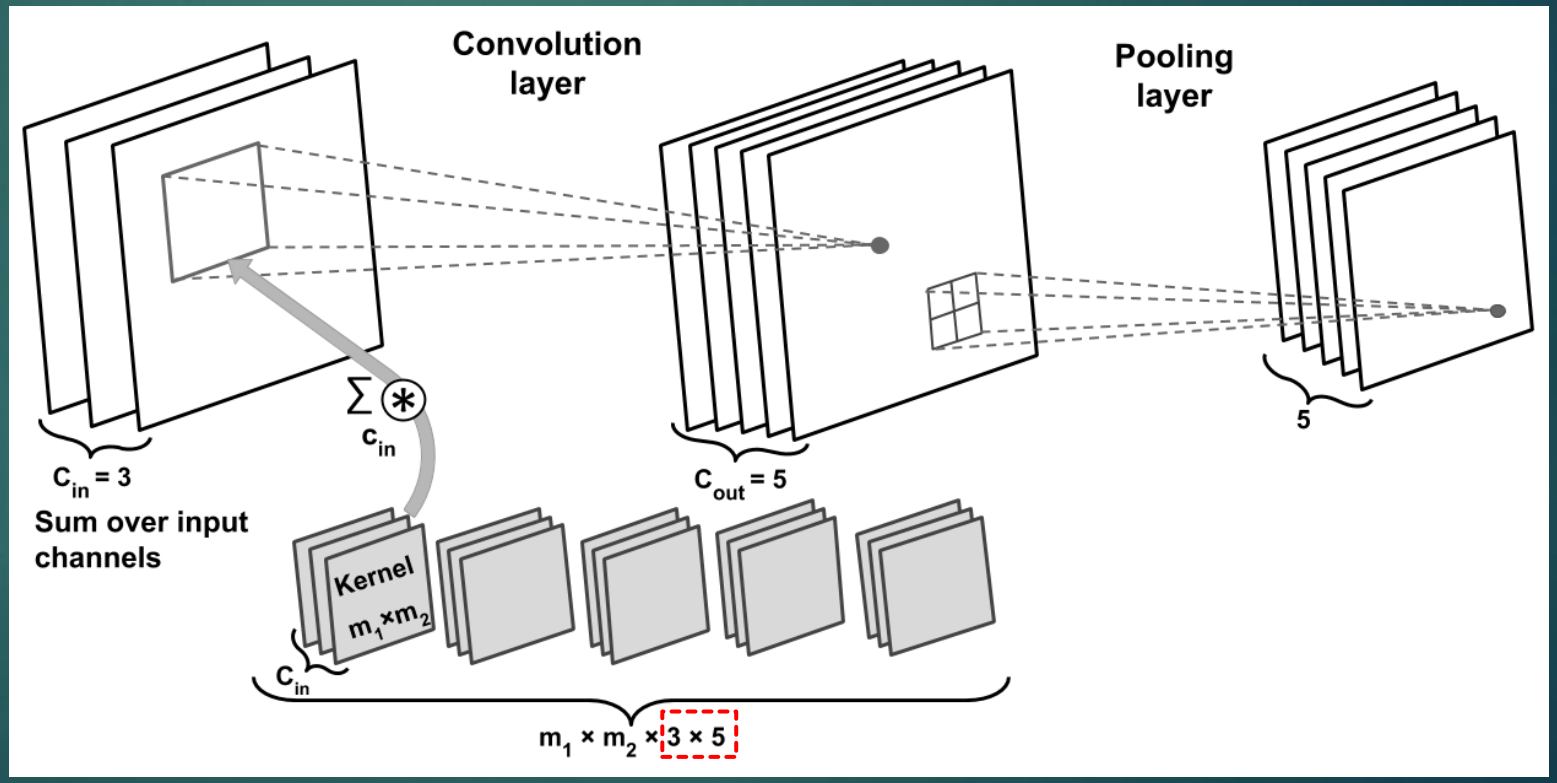
$$\Longrightarrow \begin{cases} \boldsymbol{Z}^{Conv}[:,:,k] = \sum_{c=1}^{C_{in}} \boldsymbol{W}[:,:,c,k] * \boldsymbol{X}[:,:,c] \\ \boldsymbol{Z}[:,:,k] = \boldsymbol{Z}^{Conv}[:,:,k] + b[k] \\ \boldsymbol{A}[:,:,k] = \sigma(\boldsymbol{Z}[:,:,k]) \end{cases}$$

kth Kernel, Input

kth Output, $k = 1, 2, \ldots, C_{out}$

Example of a Simple CNN

- A convolutional layer followed by a pooling layer
  - $C_{in} = 3$
  - $C_{out} = 5$



Example of a Simple CNN

- Number of trainable parameters
  - Convolutional layer

$$m_1 \times m_2 \times 3 \times 5 + 5$$

Kernel matrix Bias vector

  - Pooling layer has *no* (trainable) parameters
- Compared to implementation by a *fully connected layer*

$$(n_1 \times n_2 \times 3) \times (n_1 \times n_2 \times 5) = (n_1 \times n_2)^2 \times 3 \times 5 + n_1 \times n_2 \times 5$$

Image size Weight matrix Bias vector

Given that $m_1 < n_1$ and $m_2 < n_2$, the difference in the number of trainable parameters is significant

Regularizing an NN or CNN

- Capacity of a network
  - The level of complexity of the function that it can learn to approximate
    - Small networks → low capacity → underfitting
    - Large networks → high capacity → overfitting
  - In real-world problems, we do not know how large the network should be a priori
    - In practice, choose a capacity that is slightly larger than necessary to do well on the training dataset
    - Apply one or multiple regularization schemes for preventing overfitting to achieve good generalization performance on new data

Regularizing an NN or CNN

- L2 regularization
  - Simply add the L2 penalty of a particular layer to the loss function in PyTorch

```
>>> import torch.nn as nn
>>> loss_func = nn.BCELoss()
>>> loss = loss_func(torch.tensor([0.9]), torch.tensor([1.0]))
>>> l2_lambda = 0.001
>>> conv_layer = nn.Conv2d(in_channels=3,
...                        out_channels=5,
...                        kernel_size=5)
>>> l2_penalty = l2_lambda * sum(
...     [(p**2).sum() for p in conv_layer.parameters()]
... )
>>> loss_with_penalty = loss + l2_penalty
```

Weight: [5, 3, 5, 5]
Bias: [5]

Regularizing an NN or CNN

- Alternative way of L2 regularization
  - Setting the weight_decay parameter in a PyTorch optimizer

```
optimizer = torch.optim.SGD(
    model.parameters(),
    weight_decay=l2_lambda,
    ...
)
```

In fact, L2 regularization and weight_decay are *not* strictly identical

Regularization via Dropout

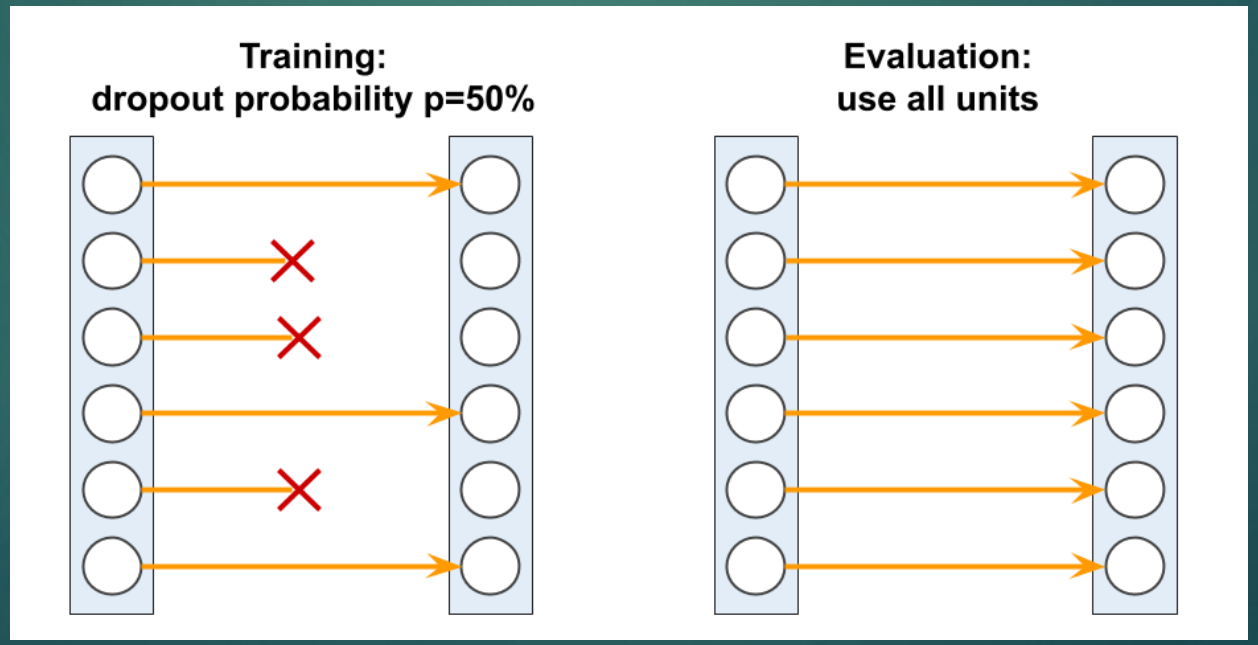
- Dropout (隨機失活) is usually applied to the hidden units of *higher* layers
  - During the training phase of an NN, a fraction of the hidden units is randomly dropped at every iteration (mini-batch) with probability p_{drop} (usually 0.5)
  - Force an NN to learn a *redundant* representation of the data
    - Interpreted as the consensus (averaging) of an ensemble of models
      - Train many models at once and compute their average predictions at test time
    - Effectively prevent overfitting

不能依賴特定的神經元組合，必須學習更具泛化性與冗餘性的通用表徵

During testing, all neurons will be used. (no dropout)

Regularization via Dropout

- PyTorch scale the activations during training (for example, by doubling the activations if $p_{drop} = 0.5$)
  - To ensure that the overall activations are *on the same scale* during training and testing



Configuring CNN layers in PyTorch

- Conveniently, use torch.nn.Sequential class to *stack* (add) different layers
  - nn.Conv2d for a 2D convolution layer
    - Assumes that inputs are in NCHW format [Batch_Size, Channels, Height, Width]
  - nn.MaxPool2d for max-pooling
  - nn.AvgPool2d for average-pooling
  - nn.Dropout for dropout

```
>>> model = nn.Sequential()
>>> model.add_module(
...     'conv1',
...     nn.Conv2d(
...         in_channels=1, out_channels=32,
...         kernel_size=5, padding=2
...     )
... )
>>> model.add_module('relu1', nn.ReLU())
>>> model.add_module('pool1', nn.MaxPool2d(kernel_size=2))
```

Check documents for optional parameters to configure a layer

Data Augmentation

Data Augmentation (DA) (資料增強)

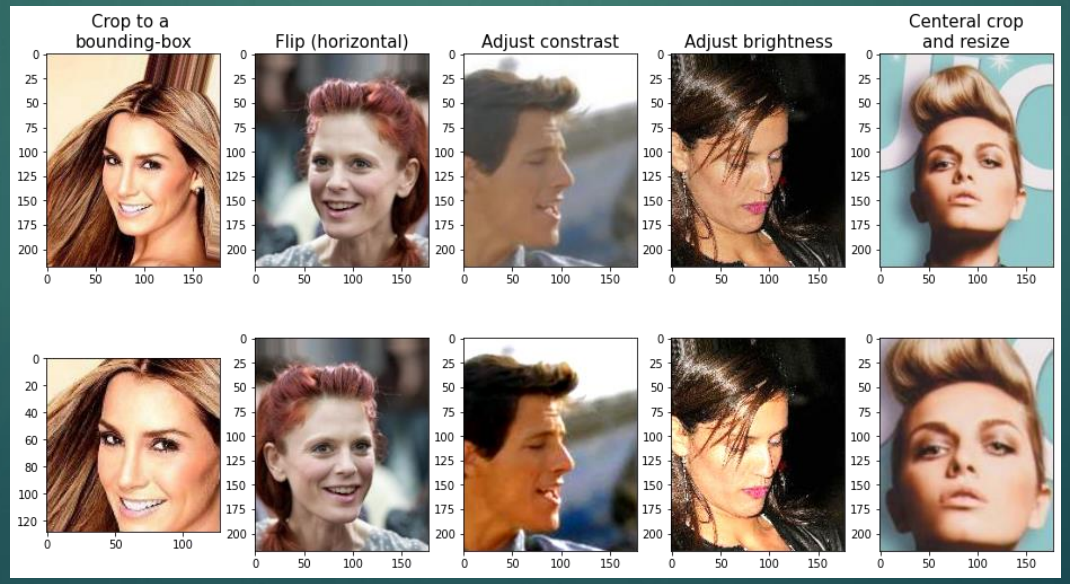
- A technique used to increase the *diversity* and *volume* of training data by generating slightly modified copies of existing data (or even artificially synthesized data)
  - Prevents overfitting
    - It stops models from simply memorizing training details
  - Improves generalization
    - It teaches models to adapt to real-world variations
  - Fixes data imbalance
    - It expands small datasets to match larger ones
  - Saves cost
    - It reduces the manual labor needed for labeling

Common DA Techniques for Image

- Flipping
  - Mirroring images horizontally or vertically
- Rotation
  - Turning images by specific degree angles
- Cropping
  - Resizing and cutting random sections of images
- Color jittering
  - Changing the brightness, contrast, or saturation

DA via torchvision.transforms

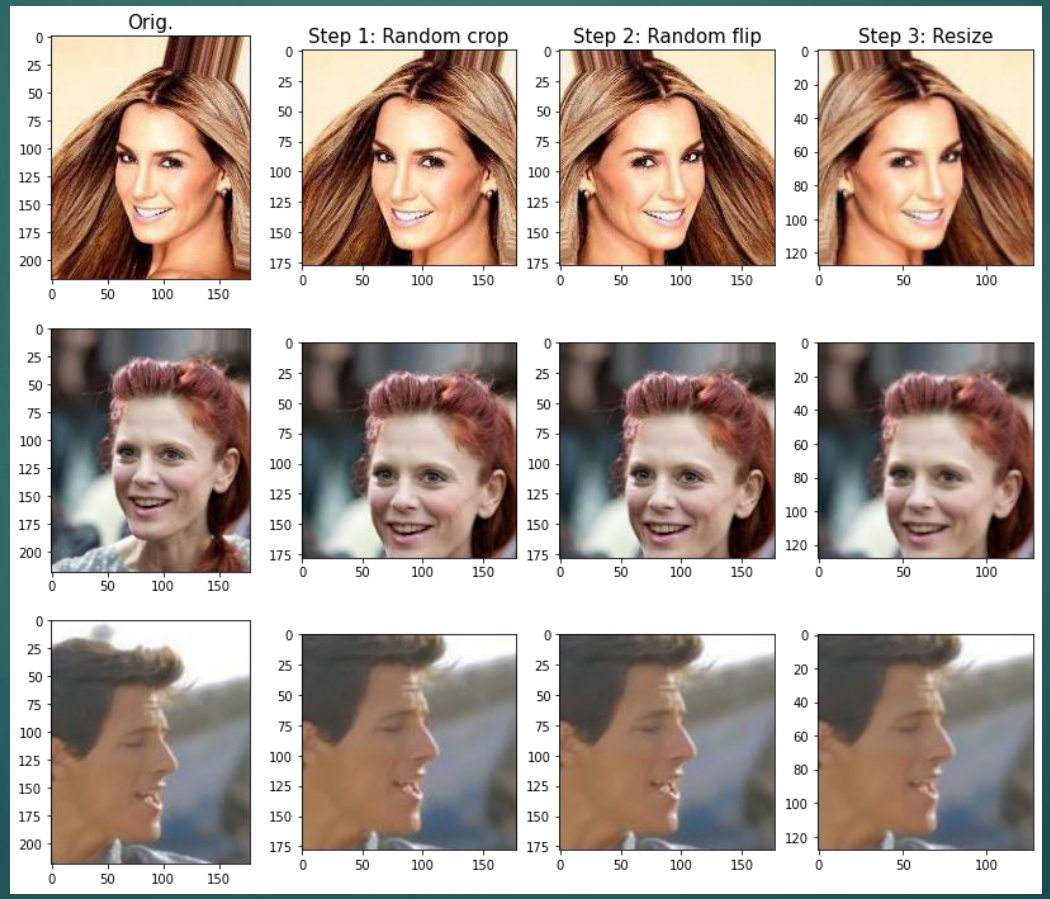
- Cropping an image to a bounding box
- Flipping an image horizontally
- Adjusting the contrast
- Adjusting the brightness
- Center-cropping an image and resizing it back to its original size



218×178

DA via torchvision.transforms

- All transformations can also be randomized which is recommended for DA during model training



Common DA Techniques for Text

- Synonym replacement
  - Swapping words with similar meanings
- Back-translation
  - Translating text to another language and back
- Random insertion/deletion
  - Adding or removing words randomly

Common DA Techniques for Audio

- Noise injection
  - Adding background sounds or static
- Time shifting
  - Moving the audio forward or backward
- Speed alteration
  - Speeding up or slowing down audio signals

Important Rules of DA

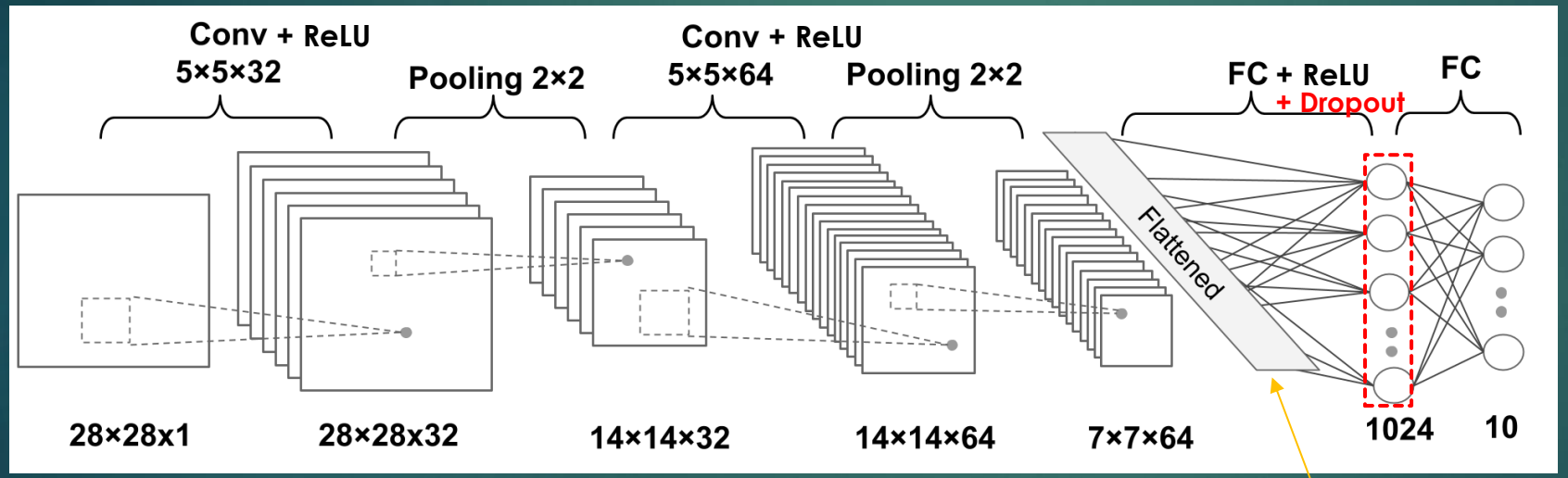
- Apply to training data only
  - Never augment validation or test data
- Preserve the label context
  - Modified data must keep its original meaning
    - For example, flipping a picture of a “left hand” turns it into a “right hand”

Building and Training Deep CNNs in PyTorch

Handwritten digit recognition

Classifying Handwritten Digits

- MNIST dataset
  - Review Lecture 7 slides, pp. 20-26
- Multilayer CNN architecture



Input: [*batchsize*×28×28×1]
Conv_1: [*batchsize*×28×28×32]
Pooling_1: [*batchsize*×14×14×32]
Conv_2: [*batchsize*×14×14×64]
Pooling_2: [*batchsize*×7×7×64]
FC_1: [*batchsize*×1024]
FC_2 and softmax layer: [*batchsize*×10]

- Batch size: *batchsize*

Input to FC layer must have rank 2:
[batchsize x input_units]

Classifying Handwritten Digits

- Tensor sizes in detail
  - Input: [64, 1, 28, 28]
  - Conv2d_1: [64, 32, 28, 28]
    - 32 kernels $(5 \times 5,\ p = 2,\ s = 1)$ (same padding mode)
  - MaxPool2d_1: [64, 32, 14, 14]
    - Pooling size $2 \times 2,\ s = 2$ (by default equal to the pooling size)
  - Conv2d_2: [64, 64, 14, 14]
    - 64 kernels $(5 \times 5,\ p = 2,\ s = 1)$ (same padding mode)
  - MaxPool2d_2: [64, 64, 7, 7]
    - Pooling size $2 \times 2,\ s = 2$ (by default equal to the pooling size)
  - Flatten: [64, 3136]
    - $64 \times 7 \times 7 = 3136$
  - FC_1: [64, 1024]
  - Dropout: [64, 1024]
    - $p_{drop} = 0.5$
  - FC_2: [64, 10] (Output 10 logits)

While we can calculate the size manually, PyTorch provides a convenient method to compute it. For example:

```
>>> x = torch.ones((4, 1, 28, 28))
>>> model(x).shape
torch.Size([4, 64, 7, 7])
```

Load Dataset and Define DataLoader

- Sample code: 08_1.py

```
65  # 2. 取得訓練資料集
66  training_data = datasets.MNIST(
67      root="MNIST",
68      train=True,
69      download=True,
70      transform=v2.Compose([v2.ToImage(), v2.ToDtype(torch.float32, scale=True)]),
71  )
72
73  valid_dataset = Subset(training_data, torch.arange(10000))
74  train_dataset = Subset(training_data, torch.arange(10000, len(training_data)))
75
76  # 3. 創建 DataLoader
77  train_dataloader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
78  valid_dataloader = DataLoader(valid_dataset, batch_size=batch_size, shuffle=False)
```

驗證集取前 10000 筆 samples
訓練集取後 50000 筆 samples

Build the Deep CNN

```
80  # 4. 定義 NN Model
81  model = nn.Sequential(
82      # 第一層卷積與池化 (輸入通道=1, 輸出通道=32)
83      nn.Conv2d(in_channels=1, out_channels=32, kernel_size=5, padding=2),
84      nn.ReLU(),
85      nn.MaxPool2d(kernel_size=2),
86
87      # 第二層卷積與池化 (輸入通道=32, 輸出通道=64)
88      nn.Conv2d(in_channels=32, out_channels=64, kernel_size=5, padding=2),
89      nn.ReLU(),
90      nn.MaxPool2d(kernel_size=2),
91
92      # 分類器
93      nn.Flatten(),
94      nn.Linear(64 * 7 * 7, 1024),
95      nn.ReLU(),
96      nn.Dropout(p=0.5),
97      nn.Linear(1024, 10) # 輸出 Logits，交由 nn.CrossEntropyLoss 處理
98  ).to(device)  # 將模型移至指定硬體
```

Choose Loss function and Optimizer

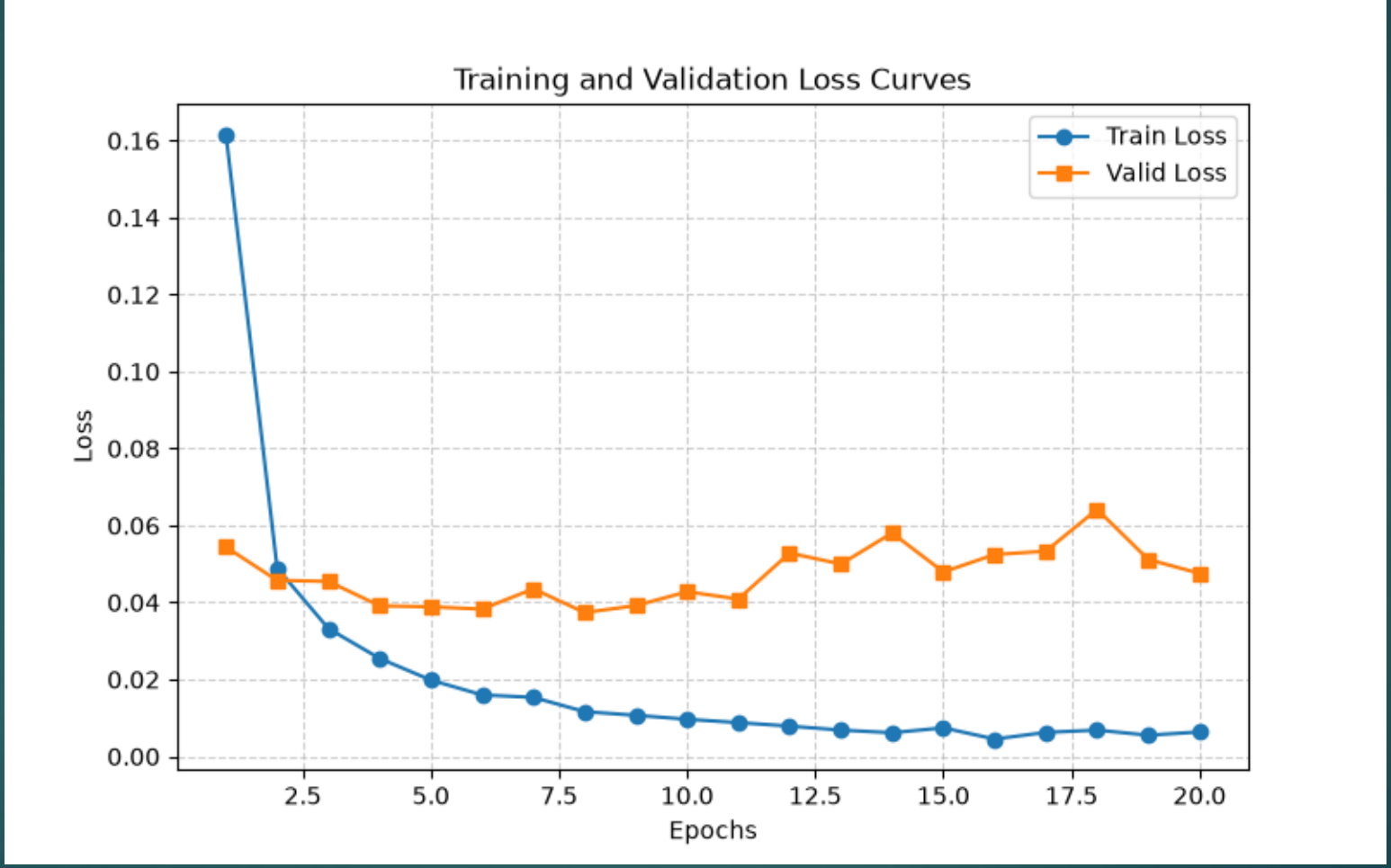
```
160    # 7. 選擇 Loss function 與 Optimizer
161    loss_fn = nn.CrossEntropyLoss()
162    optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)
```

```
46    # Hyperparameters
47    learning_rate = 1e-3
48    batch_size = 64
49    epochs = 20
```

```
Start Training...
Epoch 01 | Train Loss: 0.1617 | Valid Loss: 0.0544
Epoch 02 | Train Loss: 0.0489 | Valid Loss: 0.0457
Epoch 03 | Train Loss: 0.0332 | Valid Loss: 0.0455
Epoch 04 | Train Loss: 0.0254 | Valid Loss: 0.0391
Epoch 05 | Train Loss: 0.0198 | Valid Loss: 0.0389
Epoch 06 | Train Loss: 0.0160 | Valid Loss: 0.0383
Epoch 07 | Train Loss: 0.0153 | Valid Loss: 0.0435
Epoch 08 | Train Loss: 0.0116 | Valid Loss: 0.0374
Epoch 09 | Train Loss: 0.0107 | Valid Loss: 0.0392
Epoch 10 | Train Loss: 0.0096 | Valid Loss: 0.0428
Epoch 11 | Train Loss: 0.0088 | Valid Loss: 0.0409
Epoch 12 | Train Loss: 0.0079 | Valid Loss: 0.0528
Epoch 13 | Train Loss: 0.0068 | Valid Loss: 0.0500
Epoch 14 | Train Loss: 0.0061 | Valid Loss: 0.0581
Epoch 15 | Train Loss: 0.0074 | Valid Loss: 0.0479
Epoch 16 | Train Loss: 0.0045 | Valid Loss: 0.0524
Epoch 17 | Train Loss: 0.0062 | Valid Loss: 0.0533
Epoch 18 | Train Loss: 0.0068 | Valid Loss: 0.0642
Epoch 19 | Train Loss: 0.0055 | Valid Loss: 0.0512
Epoch 20 | Train Loss: 0.0063 | Valid Loss: 0.0475
Done!

Saved PyTorch Model State to model_CNN_MNIST.pth
```

Loss Curves


Training and Validation Loss Curves
Train Loss
Valid Loss
Loss
0.16
0.14
0.12
0.10
0.08
0.06
0.04
0.02
0.00
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0
Epochs

Classification Report

- Sample code: 08_2.py

```python
# 2. 取得 Test 資料集
test_data = datasets.MNIST(
    root="MNIST",
    train=False,
    download=True,
    transform=v2.Compose([v2.ToImage(), v2.ToDtype(torch.float32, scale=True)]),
)
```

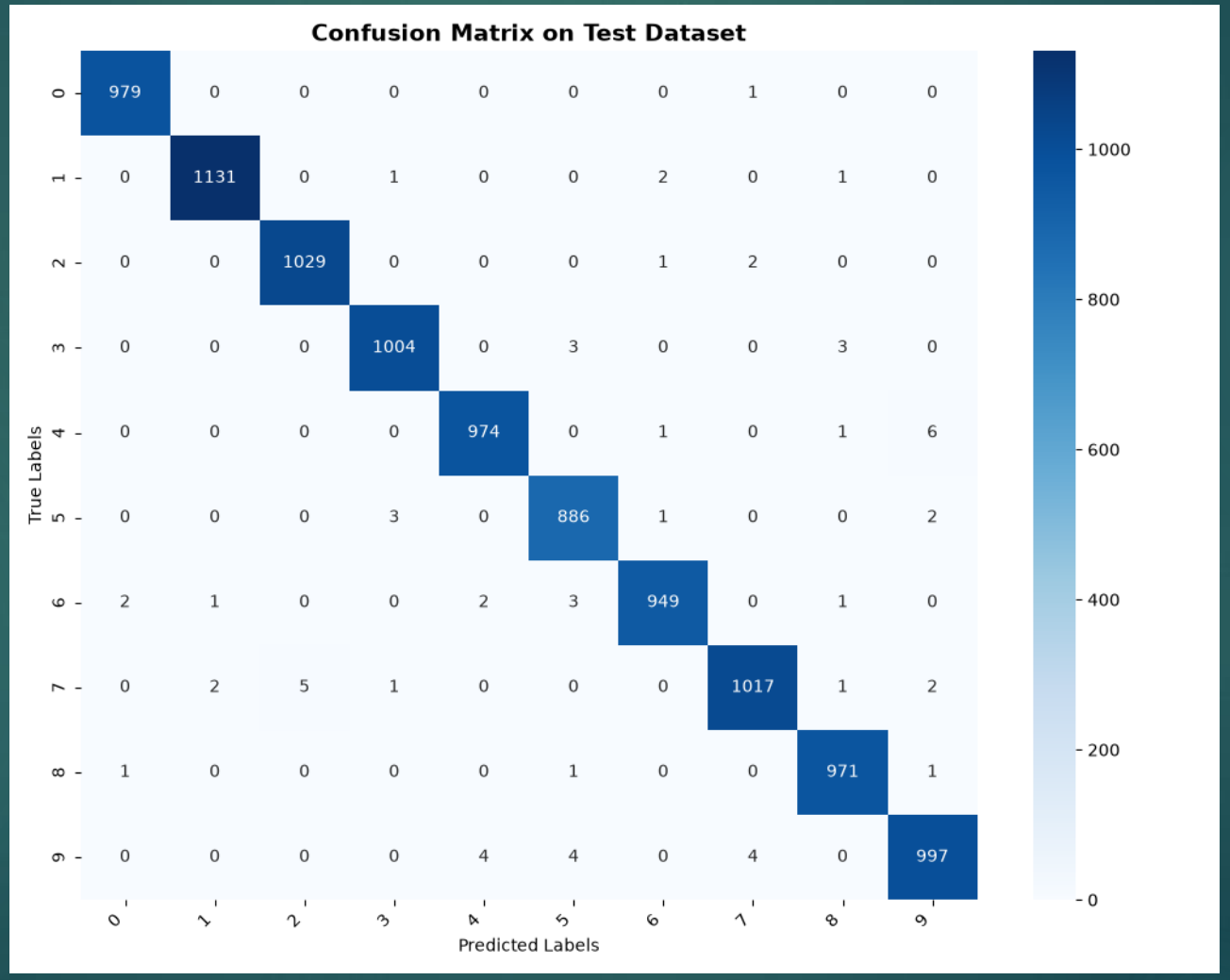
```
詳細分類報告 (Classification Report):
              precision    recall  f1-score   support

           0       1.00      1.00      1.00       980
           1       1.00      1.00      1.00      1135
           2       1.00      1.00      1.00      1032
           3       1.00      0.99      0.99      1010
           4       0.99      0.99      0.99       982
           5       0.99      0.99      0.99       892
           6       0.99      0.99      0.99       958
           7       0.99      0.99      0.99      1028
           8       0.99      1.00      0.99       974
           9       0.99      0.99      0.99      1009

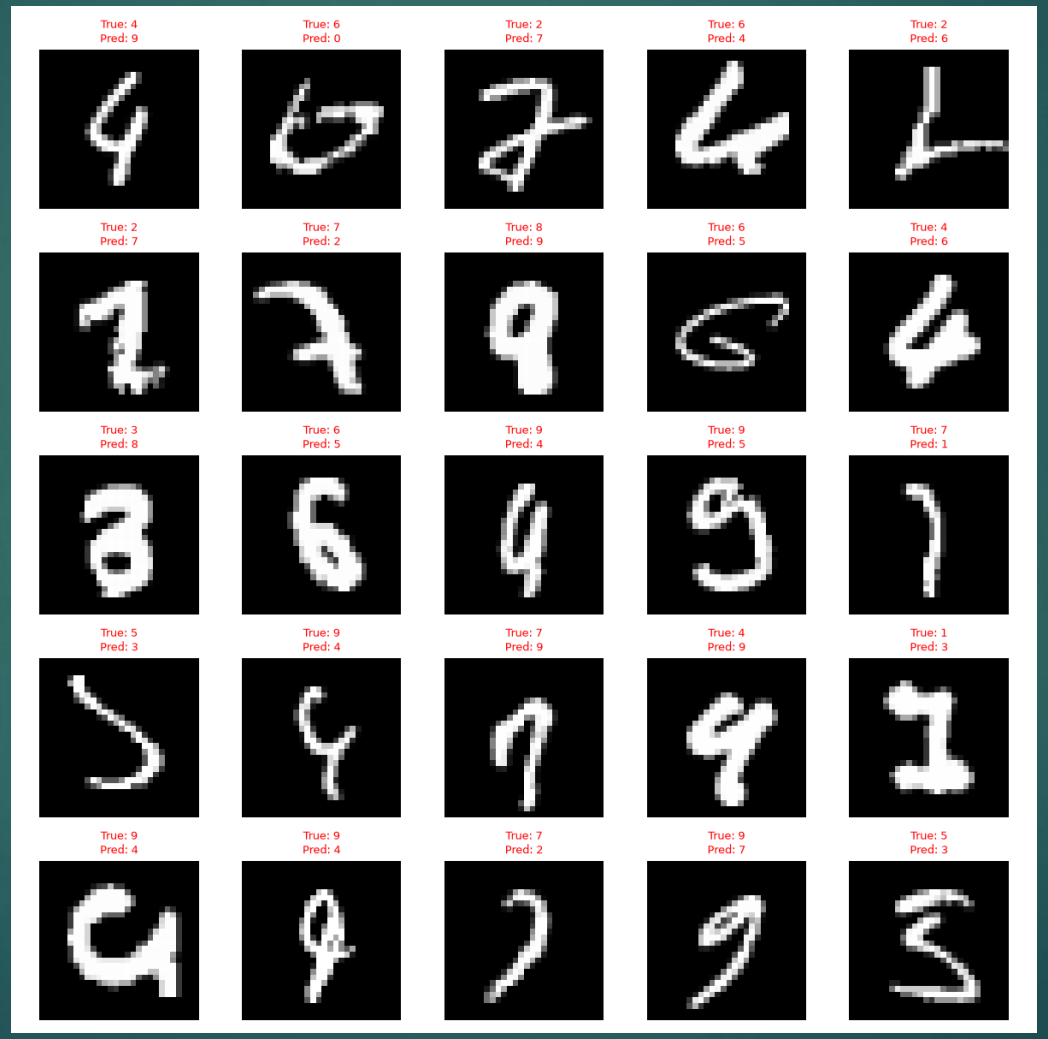
    accuracy                           0.99     10000
   macro avg       0.99      0.99      0.99     10000
weighted avg       0.99      0.99      0.99     10000

Total misclassified samples: 63 / 10000
```

Confusion Matrix



25 Misclassified Samples



Building and Training Deep CNNs in PyTorch

Smile classification from face images

Smile Classification from Face Images

- CelebA dataset
  - 202,599 RGB images of celebrities' faces
    - Train / valid / test
    - Only a small portion of the training data (16,000 training examples) will be used to speed up the training process
      - Data augmentation adopted
  - 40 binary facial attributes
    - Smiling (or not)
    - Young (or old)
    - …

40 Binary Facial Attributes

索引 (Index)	官方屬性名稱 (Attribute)	中文對照 (Chinese Translation)
0	5_o_Clock_Shadow	鬍渣（青色鬍渣）
1	Arched_Eyebrows	拱形眉（柳葉眉）
2	Attractive	有吸引力（迷人）
3	Bags_Under_Eyes	眼袋
4	Bald	禿頭（光頭）
5	Bangs	劉海
6	Big_Lips	大嘴唇（厚嘴唇）
7	Big_Nose	大鼻子
8	Black_Hair	黑髮
9	Blond_Hair	金髮
10	Blurry	模糊的（照片畫質）
11	Brown_Hair	棕髮（褐髮）
12	Bushy_Eyebrows	濃密眉毛
13	Chubby	圓潤的（豐滿/微胖）
14	Double_Chin	雙下巴
15	Eyeglasses	眼鏡
16	Goatee	山羊鬍
17	Gray_Hair	灰髮（白髮）
18	Heavy_Makeup	濃妝
19	High_Cheekbones	高顴骨
20	Male	男性（性別）
21	Mouth_Slightly_Open	微張嘴
22	Mustache	八字鬍（嘴唇上方的鬍子）
23	Narrow_Eyes	窄眼（瞇瞇眼）
24	No_Beard	沒有鬍子
25	Oval_Face	鵝蛋臉（橢圓臉）
26	Pale_Skin	蒼白皮膚（白皙皮膚）
27	Pointy_Nose	尖鼻子
28	Receding_Hairline	髮際線後退
29	Rosy_Cheeks	紅潤雙頰
30	Sideburns	鬢角
31	Smiling	微笑
32	Straight_Hair	直髮
33	Wavy_Hair	捲髮（波浪髮）
34	Wearing_Earrings	戴耳環
35	Wearing_Hat	戴帽子
36	Wearing_Lipstick	擦口紅
37	Wearing_Necklace	戴項鍊
38	Wearing_Necktie	戴領帶
39	Young	年輕

Load Dataset and Define DataLoader

- Sample code: 08_3.py
  - Module gdown is required (download files from GDrive) to load CelebA dataset
    - python -m pip install gdown

```
69  training_data = datasets.CelebA("CelebA",
70      split='train',
71      target_type='attr',
72      download=True,
73      transform=transform_train,
74      target_transform=get_smile)
75
76  valid_data = datasets.CelebA("CelebA",
77      split='valid',
78      target_type='attr',
79      download=True,
80      transform=transform,
81      target_transform=get_smile)
82
83  train_dataset = Subset(training_data, torch.arange(16000))
84  valid_dataset = Subset(valid_data, torch.arange(1000))
```

```
86  # 3. 創建 DataLoader
87  train_dataloader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
88  valid_dataloader = DataLoader(valid_dataset, batch_size=batch_size, shuffle=False)
```

訓練集只取 16000 筆 samples
驗證集只取 1000 筆 samples

Data Augmentation

```
52  get_smile = lambda attr: attr[31]
53
54  transform_train = v2.Compose([          訓練階段的資料轉換
55      v2.RandomCrop([178, 178]),          # 1. 隨機裁剪
56      v2.RandomHorizontalFlip(),          # 2. 隨機水平翻轉
57      v2.Resize([64, 64]),                # 3. 縮放到模型指定大小
58      v2.ToImage(),                       # 4. 將圖片轉換為 Tensor 圖像物件
59      v2.ToDtype(torch.float32, scale=True)  # 5. 轉換資料型態並縮放到 [0, 1]
60  ])
61
62  transform = v2.Compose([
63      v2.CenterCrop([178, 178]),      評估與測試階段 (Validation/Test)
64      v2.Resize([64, 64]),            不可以使用隨機變換，僅能使用確
65      v2.ToImage(),                   定性的中央裁剪與縮放尺寸
66      v2.ToDtype(torch.float32, scale=True)
67  ])
```

Build the Deep CNN

```
90   # 4. 定義 NN Model
91   model = nn.Sequential(
92       # 四層 Conv + ReLU + Pool + Dropout 堆疊
93       nn.Conv2d(3, 32, kernel_size=3, padding=1),   nn.ReLU(), nn.MaxPool2d(2), nn.Dropout(0.5),
94       nn.Conv2d(32, 64, kernel_size=3, padding=1),  nn.ReLU(), nn.MaxPool2d(2), nn.Dropout(0.5),
95       nn.Conv2d(64, 128, kernel_size=3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
96       nn.Conv2d(128, 256, kernel_size=3, padding=1),nn.ReLU(),
97
98       # 全域平均池化層 (GAP)
99       nn.AvgPool2d(kernel_size=8),
100      nn.Flatten(),
101
102      # 輸出層：FC + Sigmoid
103      nn.Linear(256, 1),
104      nn.Sigmoid() # 輸出 0~1 之間的機率值，搭配 nn.BCELoss 進行訓練
105  ).to(device)  # 將模型移至指定硬體
```

這個網路架構只是一個示範，並非最好的設計，有許多地方可以優化 (例如加入批次正規化 nn.BatchNorm2d)

Choose Loss function and Optimizer

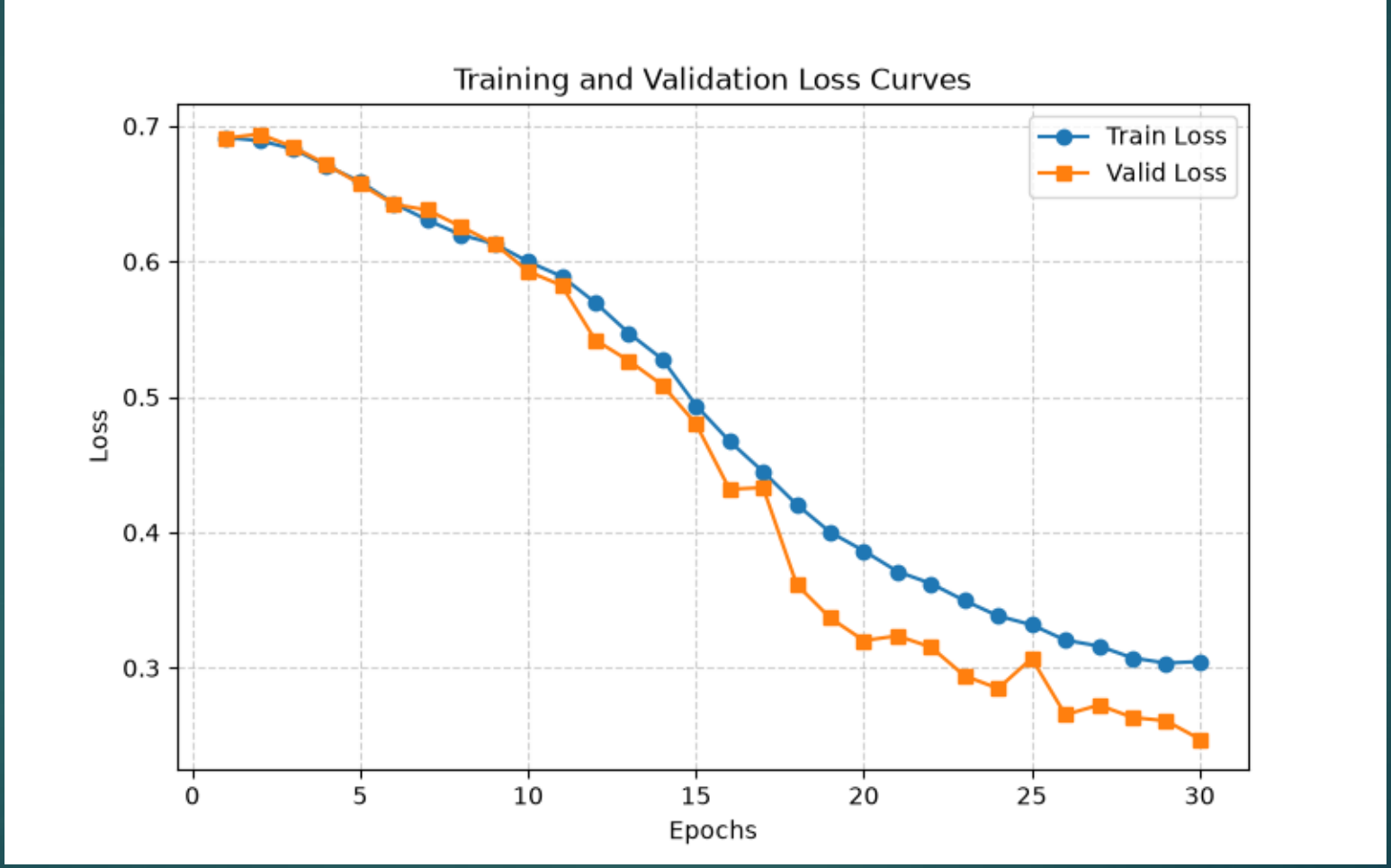
```
152    # 7. 選擇 Loss function 與 Optimizer
153    loss_fn = nn.BCELoss()
154    optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)
```

```
46    # Hyperparameters
47    learning_rate = 1e-3
48    batch_size = 32
49    epochs = 30
```

```
Start Training...
Epoch 01 | Train Loss: 0.6915 | Valid Loss: 0.6910
Epoch 02 | Train Loss: 0.6892 | Valid Loss: 0.6941
Epoch 03 | Train Loss: 0.6831 | Valid Loss: 0.6846
Epoch 04 | Train Loss: 0.6706 | Valid Loss: 0.6716
Epoch 05 | Train Loss: 0.6589 | Valid Loss: 0.6571
Epoch 06 | Train Loss: 0.6429 | Valid Loss: 0.6422
Epoch 07 | Train Loss: 0.6306 | Valid Loss: 0.6380
Epoch 08 | Train Loss: 0.6196 | Valid Loss: 0.6258
Epoch 09 | Train Loss: 0.6131 | Valid Loss: 0.6131
Epoch 10 | Train Loss: 0.5999 | Valid Loss: 0.5927
Epoch 11 | Train Loss: 0.5890 | Valid Loss: 0.5823
Epoch 12 | Train Loss: 0.5697 | Valid Loss: 0.5420
Epoch 13 | Train Loss: 0.5471 | Valid Loss: 0.5267
Epoch 14 | Train Loss: 0.5279 | Valid Loss: 0.5088
Epoch 15 | Train Loss: 0.4938 | Valid Loss: 0.4801
Epoch 16 | Train Loss: 0.4673 | Valid Loss: 0.4319
Epoch 17 | Train Loss: 0.4448 | Valid Loss: 0.4331
Epoch 18 | Train Loss: 0.4206 | Valid Loss: 0.3615
Epoch 19 | Train Loss: 0.4001 | Valid Loss: 0.3368
Epoch 20 | Train Loss: 0.3864 | Valid Loss: 0.3202
Epoch 21 | Train Loss: 0.3712 | Valid Loss: 0.3237
Epoch 22 | Train Loss: 0.3622 | Valid Loss: 0.3153
Epoch 23 | Train Loss: 0.3497 | Valid Loss: 0.2941
Epoch 24 | Train Loss: 0.3383 | Valid Loss: 0.2848
Epoch 25 | Train Loss: 0.3318 | Valid Loss: 0.3070
Epoch 26 | Train Loss: 0.3205 | Valid Loss: 0.2654
Epoch 27 | Train Loss: 0.3161 | Valid Loss: 0.2726
Epoch 28 | Train Loss: 0.3074 | Valid Loss: 0.2634
Epoch 29 | Train Loss: 0.3036 | Valid Loss: 0.2610
Epoch 30 | Train Loss: 0.3047 | Valid Loss: 0.2475
Done!

Saved PyTorch Model State to model_CelebA.pth
```

Loss Curves


Training and Validation Loss Curves
Train Loss
Valid Loss
Loss
Epochs
0.7
0.6
0.5
0.4
0.3
0
5
10
15
20
25
30

Classification Report

- Sample code: 08_4.py

```python
# 2. 取得 Test 資料集
get_smile = lambda attr: attr[31]

transform = v2.Compose([
    v2.CenterCrop([178, 178]),
    v2.Resize([64, 64]),
    v2.ToImage(),
    v2.ToDtype(torch.float32, scale=True)
])

test_data = datasets.CelebA("CelebA",
    split='test',
    target_type='attr',
    download=True,
    transform=transform,
    target_transform=get_smile)

# 3. 創建 Test DataLoader
number_of_test_samples = 10000
test_dataset = Subset(test_data, torch.arange(number_of_test_samples))
test_dataloader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)
```

測試集只取 10000 筆 samples

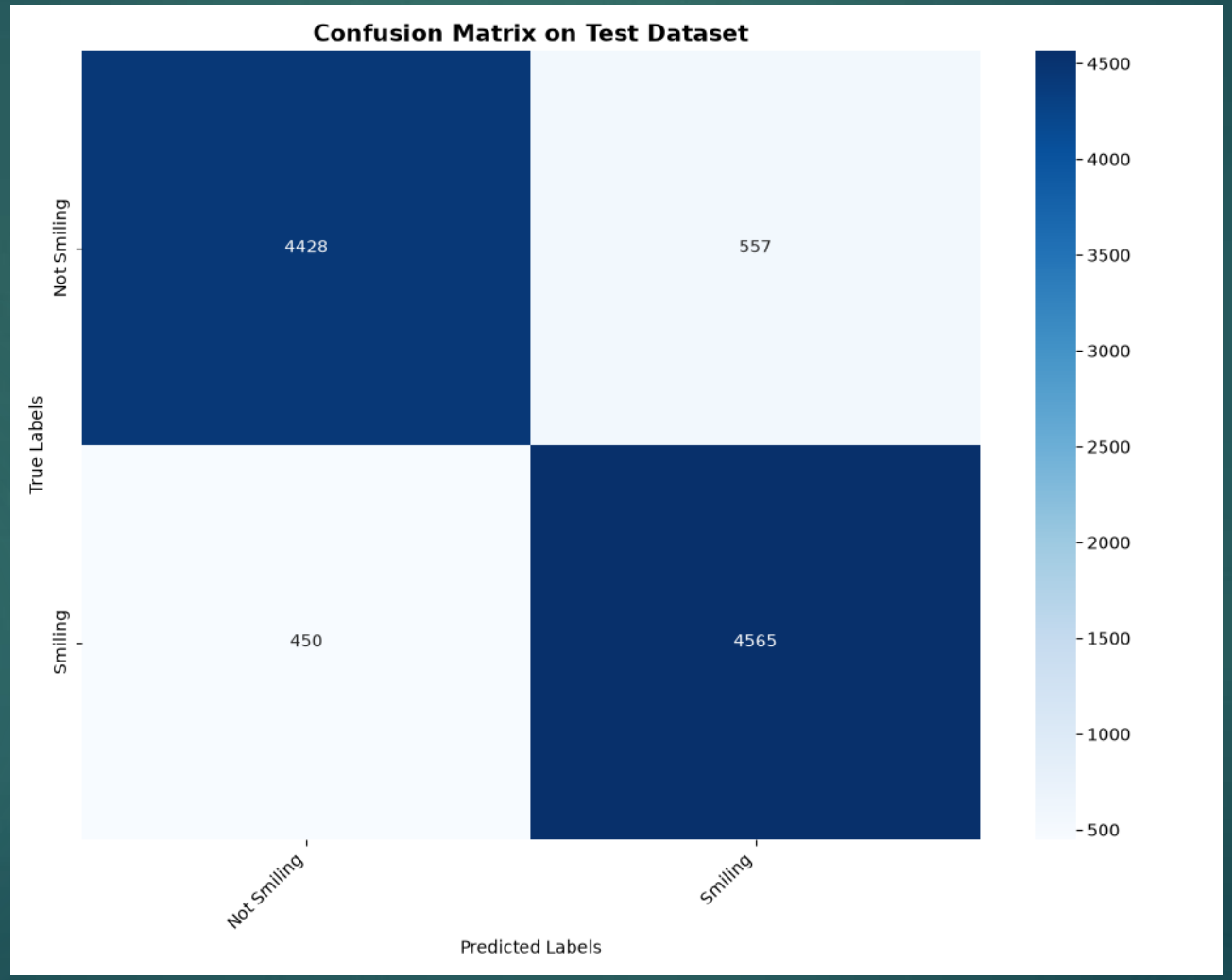
```
詳細分類報告 (Classification Report):
              precision    recall  f1-score   support

 Not Smiling       0.91      0.89      0.90      4985
     Smiling       0.89      0.91      0.90      5015

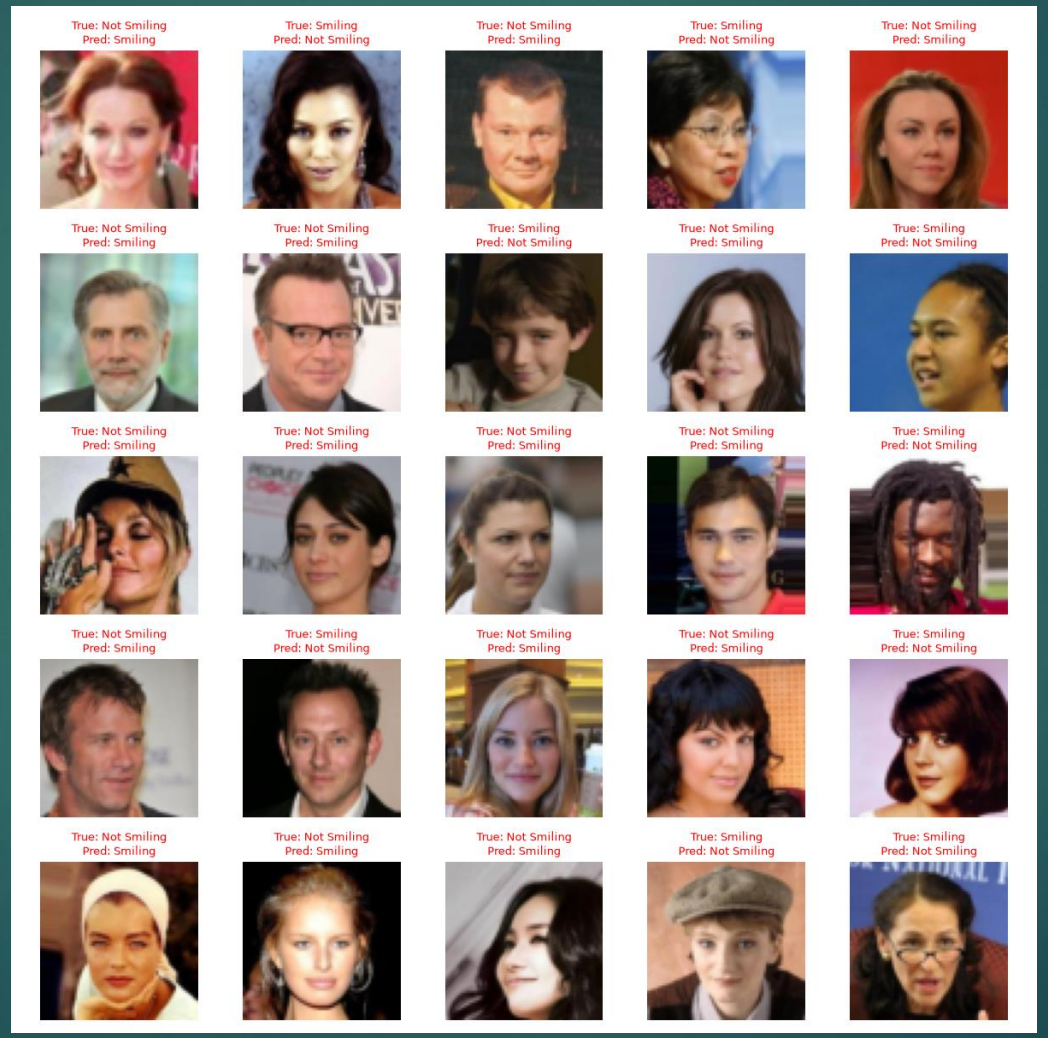
    accuracy                           0.90     10000
   macro avg       0.90      0.90      0.90     10000
weighted avg       0.90      0.90      0.90     10000

Total misclassified samples: 1007 / 10000
```

Confusion Matrix



25 Misclassified Samples



Classwork

Assignment #4

- 改善 CelebA 微笑分類任務的性能 (正確率 > 90%)
  - 方法一：調整網路架構 (see p. 60)
    - 加入批次正規化技術 (nn.BatchNorm2d)
    - 調整網路寬度或深度
    - 調整 Dropout 位置或機率
    - …
  - 方法二：使用 torchvision.models 預訓練模型進行微調 (Fine-tuning)
    - 以 ResNet-18 預訓練模型為例
      - model = torchvision.models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
      - 縮放輸入影像尺寸 (224×224)、影像正規化、修改輸出層 (對應二分類問題)

微笑分類的平均正確率通常落在 93.0% ~ 93.5% 左右 (Smiling 的正確率稍微低一點)，正確率無法再隨模型升級而改善的主因在於資料集本身的標籤錯誤率

Assignment #4

- 實作要求
  - 兩種方法皆需要實作，且資料集條件須一致
    - 僅使用 CelebA 訓練集的前 16,000 筆 samples 進行訓練
      - 可以使用資料增強技術
    - 使用 CelebA 測試集的前 10,000 筆 samples 進行測試
- 作業繳交方式
  - 總共四個檔案打包成單一 .zip 檔上傳
    - 方法一：model_1.pth (參數檔) 與 model_1_test.py (測試腳本)
    - 方法二：model_2.pth (參數檔) 與 model_2_test.py (測試腳本)
  - 測試腳本載入模型參數，請參考 Sample code: 08-4.py

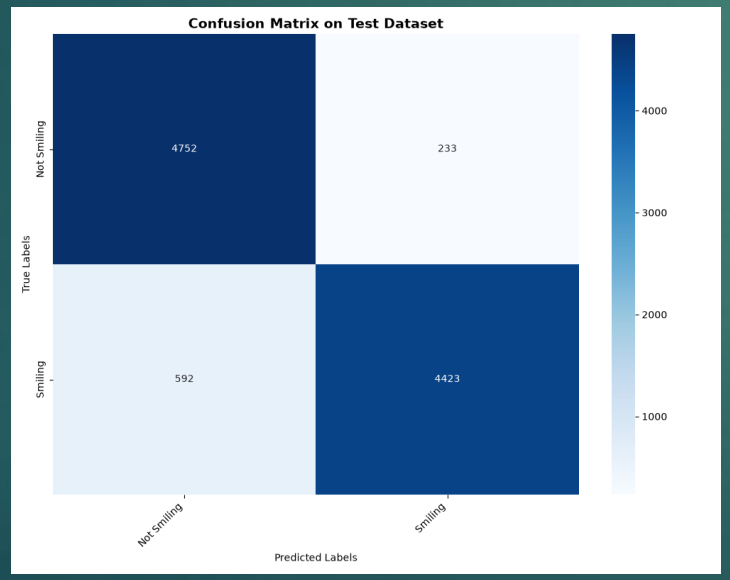
Assignment #4

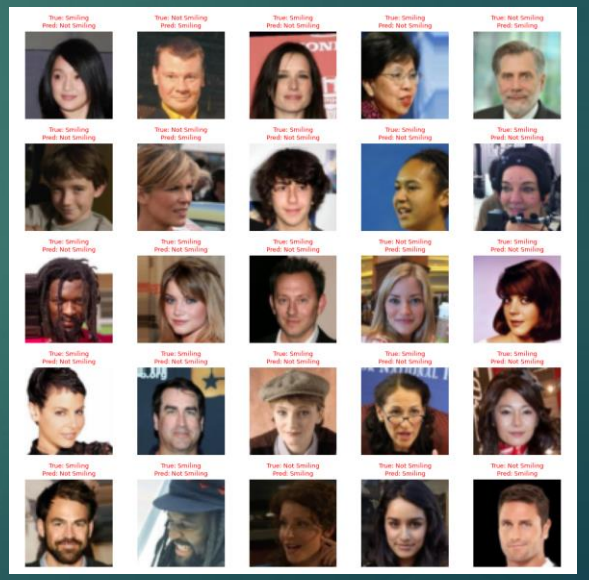
- 預期結果 (方法一)
  - Classification report
  - Confusion matrix
  - 25 misclassified samples

詳細分類報告 (Classification Report):

	precision	recall	f1-score	support
Not Smiling	0.89	0.95	0.92	4985
Smiling	0.95	0.88	0.91	5015
accuracy			0.92	10000
macro avg	0.92	0.92	0.92	10000
weighted avg	0.92	0.92	0.92	10000

Total misclassified samples: 825 / 10000





Assignment #4

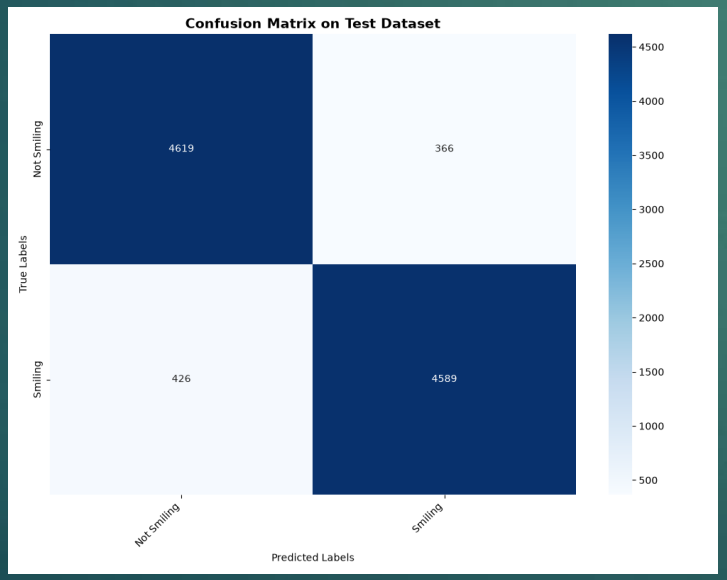
- 預期結果 (方法二)
  - Classification report
  - Confusion matrix
  - 25 misclassified samples

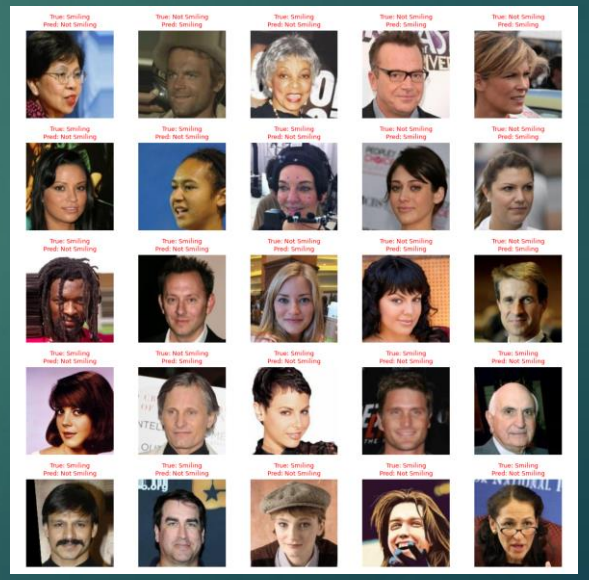
```
詳細分類報告 (Classification Report):
              precision    recall  f1-score   support

 Not Smiling       0.92      0.93      0.92      4985
     Smiling       0.93      0.92      0.92      5015

    accuracy                           0.92     10000
   macro avg       0.92      0.92      0.92     10000
weighted avg       0.92      0.92      0.92     10000

Total misclassified samples: 792 / 10000
```





Thank you